

Unidad 1: Fundamentos de JavaScript, Scope, Tipado y Memoria en Node.js

Objetivo de la unidad: Comprender la naturaleza dinámica de JavaScript, dominar la declaración moderna de variables (`const` y `let`) frente al modelo histórico (`var`), analizar el mecanismo de *hoisting* y la Zona Muerta Temporal, y dominar la gestión de datos por valor y por referencia para evitar efectos secundarios en arquitecturas modernas como Node.js y React Native.

1. Introducción y Choque Cultural: C/C# vs. JavaScript

Para un estudiante proveniente de lenguajes con tipado estático y compilación previa como C o C#, el principal cambio de paradigma al introducirse en JavaScript (JS) radica en la ausencia de una fase de compilación estricta orientada a tipos y en cómo el motor gestiona las variables en tiempo de ejecución.

Mientras que en C# el compilador verifica tipos y ámbitos de alcance (*scopes*) antes de generar el código intermedio (bytecode) o el ejecutable, JavaScript es un lenguaje interpretado / compilado en tiempo de ejecución (JIT - *Just-In-Time*) donde las reglas de tipado y la memoria operan bajo una lógica fuertemente dinámica.

2. Tipado Estático vs. Tipado Dinámico

En C y C#, el tipo de dato está fuertemente atado a la **variable** durante el tiempo de compilación. En JavaScript, el tipo de dato pertenece exclusivamente al **valor** alojado en tiempo de ejecución, lo que permite que una misma variable contenga distintos tipos a lo largo de la ejecución.

Característica	C / C# (Tipado Estático)	JavaScript (Tipado Dinámico)
Verificación de tipos	Estática (Tiempo de compilación)	Dinámica (Tiempo de ejecución)
Asociación del tipo	Atado a la variable (<code>int x = 10;</code>)	Atado al valor (<code>let x = 10;</code>)
Reasignación de tipo	No permitida (Error de compilación)	Permitida libremente durante el runtime

Característica	C / C# (Tipado Estático)	JavaScript (Tipado Dinámico)
Coerción de tipos	Estricta / Explicita	Implícita / Débil (Autoconversión)

Comportamiento dinámico y coerción de tipos en Node.js

JavaScript realiza conversión automática de tipos (*type coercion*) de forma implícita cuando interactúan operandos de distinta naturaleza:

```
// 1. Declaración e inferencia dinámica de tipo
let x = 42;
console.log("Valor:", x, "| Tipo:", typeof x); // Output: Valor: 42 | Tipo: number
```

```
// 2. Reasignación de tipo en tiempo de ejecución
x = "Hola Mundo";
console.log("Valor:", x, "| Tipo:", typeof x); // Output: Valor: Hola Mundo | Tipo: string
```

```
x = true;
console.log("Valor:", x, "| Tipo:", typeof x); // Output: Valor: true | Tipo: boolean
```

```
// 3. Coerción implícita de tipos
let sumaCoercitiva = "5" + 2;
console.log("Resultado '5' + 2:", sumaCoercitiva, "| Tipo:", typeof sumaCoercitiva); // Output: 52 | string
```

```
let restaCoercitiva = "5" - 2;
console.log("Resultado '5' - 2:", restaCoercitiva, "| Tipo:", typeof restaCoercitiva); // Output: 3 | number
```

3. Operadores de Igualdad y Coerción de Tipos

Debido a la coerción implícita, el uso inadecuado de operadores de comparación en JavaScript produce resultados ambiguos y propensos a errores lógicos en producción.

Operador	Nombre	Comportamiento	Ejemplo
==	Igualdad Débil	Convierte los tipos antes de comparar (Aplica coerción)	5 == "5" → true

Operador	Nombre	Comportamiento	Ejemplo
===	Igualdad Estricta	Compara Tipo y Valor sin realizar conversión	5 === "5" → false
!=	Desigualdad Débil	Aplica coerción de tipos antes de evaluar desigualdad	5 != "5" → false
!==	Desigualdad Estricta	Evalúa si los valores o los tipos son diferentes	5 !== "5" → true

// Ejemplos de coerción incoherente con == (EVITAR EN CÓDIGO DE PRODUCCIÓN)

```
console.log(0 == false); // true
console.log("" == false); // true
console.log(null == undefined); // true
```

// Buenas Prácticas: Uso exclusivo de === (EVALUACIÓN SEGURA)

```
console.log(0 === false); // false
console.log("" === false); // false
console.log(5 === "5"); // false
```

4. Declaración de Variables: const, let y por qué EVITAR var

Un desarrollador habituado a C/C# asume que toda variable vive únicamente dentro de las llaves { } que la contienen. En el JavaScript histórico existía únicamente la palabra clave var, la cual ignoraba el ámbito de bloque (*block scope*) y se "fugaba" al ámbito de función o global.

Con la llegada del estándar ES6+ (JavaScript moderno), se introdujeron los modificadores let y const para corregir esta deficiencia.

Criterio	var (Obsoleto)	let	const
Alcance (Scope)	De Función o Global	De Bloque { }	De Bloque { }

Criterio	var (Obsoleto)	let	const
Re-declaración	Permitida en el mismo ámbito	Error de Sintaxis (<i>SyntaxError</i>)	Error de Sintaxis (<i>SyntaxError</i>)
Re-asignación	Permitida	Permitida	No permitida
Hoisting	Sí (Inicializada como undefined)	Sí (Permanencia en TDZ)	Sí (Permanencia en TDZ)

El problema histórico de la "fuga" de variables con var

```
// Ejemplo del problema con 'var':
if (true) {
  var fuga = "Me escapé del bloque";
}
// En C#, 'fuga' no existiría aquí. En JS histórico con 'var':
console.log(fuga); // Imprime: "Me escapé del bloque" (Ignora las llaves del if)
```

```
// Solución moderna con let y const (Comportamiento esperado):
if (true) {
  let contador = 10;
  const limite = 100;
}
// console.log(contador); // ReferenceError: contador is not defined
```

Regla de Oro en Producción: Declarar siempre por defecto con `const`. Si la variable requiere cambiar de valor durante el flujo (por ejemplo, acumuladores o contadores de bucles), utilizar `let`. Evitar completamente el uso de `var`.

Inmutabilidad de referencia vs. Inmutabilidad de contenido en const

En JavaScript, la palabra reservada `const` no define un valor inmutable en su totalidad, sino una **referencia no reasignable** a una posición de memoria. Si la variable almacena un tipo complejo (objeto o array), las propiedades o elementos internos sí pueden mutar.

```
const edad = 30;
// edad = 31; // TypeError: Assignment to constant variable.
```

```
const auto = { marca: "Volkswagen", modelo: "Suran", anio: 2011 };
```

```
// PERMITIDO: Mutación de propiedades internas
auto.modelo = "Gol";
auto.color = "Gris";
console.log(auto); // Output: { marca: 'Volkswagen', modelo: 'Gol', anio: 2011, color: 'Gris' }
```

```
// NO PERMITIDO: Reasignación de la referencia en memoria
// auto = { marca: "Ford" }; // TypeError: Assignment to constant variable.
```

5. El Concepto de Hoisting ("Elevación") y TDZ

El motor de JavaScript no ejecuta el código de forma puramente lineal. Antes de la ejecución, realiza una fase de compilación JIT donde registra las declaraciones de variables y funciones dentro del scope correspondiente.

Hoisting con var

Las variables declaradas con var son elevadas al inicio de la función o script y son inicializadas automáticamente con el valor undefined.

```
console.log("Valor antes de declarar:", nombre); // Output: undefined (No arroja error)
var nombre = "Carlos";
console.log("Valor después de declarar:", nombre); // Output: Carlos

// Interpretación interna del motor de JS:
// var nombre;
// console.log("Valor antes de declarar:", nombre);
// nombre = "Carlos";
```

Hoisting con let y const: La Zona Muerta Temporal (TDZ)

Las variables declaradas con let y const también sufren *hoisting*, pero **no son inicializadas**. Quedan en un estado reservado denominado Zona Muerta Temporal (TDZ - *Temporal Dead Zone*) desde el inicio del bloque hasta que el flujo alcanza la línea exacta de su declaración.

```
// Intentar acceder a la variable dentro de la TDZ genera un error explícito:
// console.log(apellido); // ReferenceError: Cannot access 'apellido' before initialization
let apellido = "Gómez";
```

Hoisting en Funciones

- **Function Declaration:** Se eleva la definición completa de la función. Puede ser invocada antes de la línea donde fue escrita.
saludar(); // Funciona correctamente
function saludar() { console.log("Hola!"); }
- **Function Expression:** Si la función se asigna a una variable (especialmente con var, let o const), solo se eleva la variable. Invocarla antes de su asignación genera un error.
// despedir(); // TypeError: despedir is not a function (si es var) o ReferenceError (si es let/const)
var despedir = function() { console.log("Chau!"); };

6. Tipos Primitivos vs. Objetos: Pasaje por Valor y por Referencia

Entender la gestión de memoria es fundamental para evitar efectos secundarios imprevistos al trabajar con estados en aplicaciones Node.js y frameworks como React Native.

Tipos Primitivos (Inmutables por Valor)

JavaScript posee 7 tipos primitivos: number, string, boolean, undefined, null, bigint y symbol. Al asignar o copiar un tipo primitivo, el motor reserva un nuevo espacio en memoria e independiza los valores.

```
let a = 10;
let b = a; // Copia el VALOR 10 en una nueva casilla de memoria
b = 20;
```

```
console.log("a:", a); // Output: 10 (Permanecen intactos)
console.log("b:", b); // Output: 20
```

```
function incrementar(numero) {
  numero = numero + 1;
  return numero;
}
let contador = 5;
incrementar(contador); // Se pasa una copia del valor primitivo
console.log("contador:", contador); // Output: 5
```

Tipos Complejos / Objetos (Pasaje por Referencia)

Estructuras compuestas como Objetos literales ({}), Arrays ([]) y Funciones almacenan direcciones de memoria (punteros implícitos). Al asignar un objeto a otra variable, no se duplicará el contenido, sino que ambas variables apuntarán exactamente a la misma posición física de memoria.

```
const usuario1 = { nombre: "Osvaldo", rol: "Profe" };
const usuario2 = usuario1; // 'usuario2' recibe la REFERENCIA en memoria
```

```
usuario2.rol = "Coordinador"; // Se muta la propiedad a través de 'usuario2'
```

```
// ¡Ambas variables reflejan la modificación por compartir la referencia!
console.log("usuario1:", usuario1.rol); // Output: Coordinador
console.log("usuario2:", usuario2.rol); // Output: Coordinador
```

```
function modificarLegajo(estudiante) {
  estudiante.legajo = 999;
}
modificarLegajo(usuario1);
console.log("usuario1 legajo:", usuario1.legajo); // Output: 999
```

Comparación de Objetos en Memoria

El operador de comparación sobre estructuras complejas evalúa si ambas variables apuntan a la misma dirección física de memoria, independientemente de si su contenido léxico es idéntico.

```
let objA = { id: 1 };
let objB = { id: 1 };
console.log(objA === objB); // false (Ocupan distintas posiciones de memoria)

let objC = objA;
console.log(objA === objC); // true (Comparten la misma dirección)
```

7. Actividad Práctica de Fijación: "Pensá como el Motor de JS"

Instrucciones: Analice los siguientes snippets de código, determine el resultado o error generado y luego verifíquelo ejecutándolo mediante Node.js en la terminal integrada de VS Code.

Ejercicio 1: Scope, Hoisting y TDZ

```
function probarVariables() {
  console.log("A:", x);
  if (true) {
    var x = 100;
    let y = 200;
  }
  console.log("B:", x);
  console.log("C:", y);
}
probarVariables();
```

Ejercicio 2: Variable Shadowing con let

```
function probarScope() {
  let x = 1;
  if (true) {
    let x = 2;
    console.log("Dentro del if:", x);
  }
  console.log("Fuera del if:", x);
}
probarScope();
```

Ejercicio 3: Mutación de Objetos y Arrays en const

```
const configuracion = { puerto: 3000, modo: "desarrollo" };
configuracion.puerto = 8080;
console.log("Puerto:", configuracion.puerto);
```

```
const frutas = ["Manzana", "Banana"];
const copiaFrutas = frutas;
copiaFrutas.push("Naranja");

console.log("Original longitud:", frutas.length);
console.log("Copia longitud:", copiaFrutas.length);

// ¿Qué ocurre en el siguiente paso?
// configuracion = { puerto: 8080, modo: "produccion" };
```

Ejercicio 4: Valor vs. Referencia

```
let contadorA = 10;
let contadorB = contadorA;
contadorB = 20;

let listaA = [1, 2, 3];
let listaB = listaA;
listaB.push(4);

console.log("contadorA:", contadorA);
console.log("listaA:", listaA);
```

Ejercicio 5: Coerción y Comparación de Iguales

```
let obj1 = { id: 1 };
let obj2 = { id: 1 };
let obj3 = obj1;

console.log("Comparación 1:", obj1 == obj2);
console.log("Comparación 2:", obj1 === obj3);
console.log("Comparación 3:", "5" == 5);
console.log("Comparación 4:", "5" === 5);
```